# Софтуерна спецификация и системни изисквания за платформа „Фермерски каталог“

## Архитектурен преглед и системна концепция

Дигитализацията на селскостопанския сектор и изграждането на специализирани платформи за директна търговия между земеделски производители, крайни потребители и B2B клиенти (ресторанти и търговски вериги) изисква скалируема, сигурна и поддържаема софтуерна архитектура[1]. Архитектурният проект на платформата „Фермерски каталог“ се базира на технологичния стек Java 21 и Spring Framework екосистемата (Spring Boot 3, Spring Data JPA, Spring Security, Spring Web), с релационна база данни MySQL 8.x[1]. За реализацията на клиентския интерфейс се предвижда изграждане на самостоятелно Single Page Application (SPA) с React, както и интеграция на външна автентикация чрез Google OAuth 2.0[1].

Системата е проектирана съгласно принципите на напълно разделена бекенд и фронтенд архитектура (Decoupled Architecture)[1]. Бекенд модулът функционира като безстатусен (stateless) RESTful уеб услуга, обработваща JSON заявки и издаваща криптографски подписни JSON Web Tokens (JWT) за идентификация[1]. Този архитектурен избор осигурява независимо скалиране на отделните слоеве, възможност за паралелна разработка и бъдещо разширение към мобилни приложения без необходимост от промяна в основната бизнес логика.

### Архитектура на сигурността и Google OAuth 2.0 интеграция

Подсистемата за сигурност се управлява от Spring Security 6[1]. Входът в платформата поддържа два паралелни механизма: традиционен вход с имейл и хеширана парола, и социален вход чрез Google OAuth 2.0[1].

При социалната автентикация React клиентът инициира диалог с Google Identity Services и получава подписан id_token. Този токен се изпраща към специализиран бекенд ендпоинт (/api/auth/google), където компонентът GoogleIdTokenVerifier от Spring Security извършва криптографска верификация на външния подпис[1]. При успешна проверка системата извлича потребителския имейл и извършва справка в базата данни. Ако потребителят се регистрира за първи път, автоматично се генерира нов профил в таблицата users с роля по подразбиране BUYER[2]. На следващия етап Spring Security издава вътрешен JWT токен с определен срок на валидност, съдържащ потребителското ID и присвоените системни роли (BUYER, FARMER, ADMIN), който контролира достъпа до защитените REST маршрути[1].

## Модел на данните и спецификация на релационната база данни

Миграцията от документално-ориентиран модел (NoSQL) към релационна база данни

MySQL 8.x налага пълна нормализация на данните до Трета нормална форма (3NF)[2]. Релационният модел се състои от 7 свързани таблици, осигуряващи релационен интегритет и бързо филтриране[2].

## Структура на ентитетите и техните релации

| Таблица | Основно предназначение | Релации и първични / външни ключове |
|---|---|---|
| users | Управление на системните акаунти, роли и автентикационни данни[2]. | PK: id (BIGINT)[2]. Unique Index: email[2]. |
| regions | Нормализирана географска номенклатура на регионите[2]. | PK: id (BIGINT)[2]. Unique Index: name[2]. |
| farm_profiles | Публичен профил и подробни данни за стопанството[2]. | FK: user_id -> users(id) (1:1, Unique)[2]. FK: region_id -> regions(id) (Many-to-One)[2]. |
| categories | Номенклатура на продуктовите категории[2]. | PK: id (BIGINT)[2]. Unique Index: name[2]. |
| products | Търговски регистър на предлаганите селскостопански артикули[2]. | FK: farm_id -> farm_profiles(id) (CASCADE)[2]. FK: category_id -> categories(id) (RESTRICT)[2]. |
| product_seasonality | Дефиниция на месеците на наличност и зрялост за всеки продукт[2]. | Composite PK: (product_id, month_number)[2]. FK: product_id -> products(id) (CASCADE)[2]. |
| inquiries | B2B и B2C запитвания и търговски заявки[2]. | FK: buyer_id -> users(id)[2]. FK: product_id -> products(id) (CASCADE)[2]. |

## DDL Скрипт за генериране на релационната схема

Скриптът за създаване на структурата съдържа декларации за интегритет, автоматично

обновяване на времеви печати и оптимистично заключване чрез версия[2]:

```
SQL
CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    role ENUM('BUYER', 'FARMER', 'ADMIN') NOT NULL DEFAULT 'BUYER',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE regions (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE farm_profiles (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL UNIQUE,
    region_id BIGINT NOT NULL,
    farm_name VARCHAR(150) NOT NULL,
    description TEXT,
    address VARCHAR(255) NOT NULL,
    contact_email VARCHAR(100) NOT NULL,
    image_url VARCHAR(255),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (region_id) REFERENCES regions(id)
);

CREATE TABLE categories (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(255)
);

CREATE TABLE products (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
    farm_id BIGINT NOT NULL
    category_id BIGINT NOT NULL
    name VARCHAR(150) NOT NULL
    description TEXT
    price DECIMAL(10,2) NOT NULL
    unit ENUM('KG','G','LITER','PIECE') NOT NULL
    stock_quantity DECIMAL(10,2) NOT NULL DEFAULT 0.00
    is_available BOOLEAN NOT NULL DEFAULT TRUE
    image_url VARCHAR(255)
    version BIGINT NOT NULL DEFAULT 0
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
    FOREIGN KEY (farm_id) REFERENCES farm_profiles(id) ON DELETE CASCADE
    FOREIGN KEY (category_id) REFERENCES categories(id) RESTRICT
);

CREATE TABLE product_seasonality (
    product_id BIGINT NOT NULL
    month_number TINYINT NOT NULL CHECK (month_number BETWEEN 1 AND 12)
    PRIMARY KEY (product_id, month_number)
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
);

CREATE TABLE inquiries (
    id BIGINT AUTO_INCREMENT PRIMARY KEY
    buyer_id BIGINT NOT NULL
    product_id BIGINT NOT NULL
    requested_quantity DECIMAL(10,2) NOT NULL
    message TEXT
    status ENUM('PENDING','ACCEPTED','REJECTED') DEFAULT 'PENDING'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    FOREIGN KEY (buyer_id) REFERENCES users(id)
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
);
```

## Бизнес правила, правила за валидация и ограничения на сигурността

Защитата на данните и коректността на бизнес процесите се гарантират от многослойна система за проверка, обхващаща DTO слоя, сервизните компоненти и релационните ограничения в базите данни[4].

### Входящи валидации на ниво DTO (Jakarta Bean Validation)

Всеки входящ JSON обект се подлага на декларативна валидация в контролерите на Spring Boot чрез анотации от jakarta.validation.constraints.*[4]:

| DTO Клас | Поле | Приложени анотации | Бизнес изискване и съобщение за грешка |
|---|---|---|---|
| UserRegisterDTO | email | @NotBlank, @Email[4] | Задължителен имейл формат. Уникалност през UserRepository.existsByEmail()[4]. |
| UserRegisterDTO | password | @NotBlank, @Size(min=8, max=64), @Pattern[4] | Задължителна дължина и наличие на поне една главна буква, малка буква и цифра[4]. |
| UserRegisterDTO | phoneNumber | @NotBlank, @Pattern(regexp="^(\\+359\|0)[0-9]{9}$")[4] | Валидация на национален български формат за мобилен/фиксиран телефон[4]. |
| ProductDTO | name | @NotBlank, @Size(min=2, max=150)[4] | Дължината на името трябва да бъде между 2 и 150 символа[4]. |
| ProductDTO | price | @NotNull, @DecimalMin("0.01"), @Digits(integer=8, fraction=2)[4] | Задължителна стойност над 0.00 лв. с точност до втория десетичен знак[4]. |
| ProductDTO | stockQuantity | @NotNull, @DecimalMin("0.00")[4] | Не се допуска въвеждане на отрицателни количества[4]. |

| | | | |
|---|---|---|---|
| ProductDTO | seasonality | @NotEmpty, @Min(1), @Max(12)[4] | Изисква списък от поне един валиден месечен номер (от 1 до 12)[4]. |
| InquiryDTO | requestedQuantity | @NotNull, @Positive[4] | Заявеното количество за търговска оферта трябва да е строго положително[4]. |

## Бизнес правила, ресурсен достъп и управление на конкурентността

Проектът прилага следните задължителни бизнес ограничения:
Строго спазване на връзката едно-към-едно гарантира, че всеки потребител с роля FARMER може да притежава точно един профил на стопанство[3]. Потребители с роля BUYER нямат системно право да създават или администрират фермерски профили[4].
Контролът върху собствеността на ресурсите (Resource Ownership) се извършва динамично[4]. Когато фермер опита да актуализира или изтрие продукт през маршрутите /api/products/{id}, бекендът извлича идентичността на автентикирания субект от SecurityContextHolder и я съпоставя с farm.user.id на записа от базата данни[4]. При установено разминаване Spring Security прекратява обработката и връща статус 403 Forbidden[4].
Търговските запитвания съдържат логическа защита, която забранява на фермер да изпраща запитване към продукти, произведени от собственото му стопанство[4].
Масовите операции по актуализация на наличности и цени (PUT /api/products/bulk) се изпълняват в капсулирана DB транзакция с декларативна анотация @Transactional[1]. При възникване на грешка във валидацията на който и да е артикул от изпратения списък, Spring сервизният слой задейства пълен откат (Rollback), което гарантира, че в базата данни няма да бъдат записва частично обновени данни[4].
За предотвратяване на грешки от застъпващо се презаписване на данни при синхронни промени от различни сесии (Race Conditions), ентитетът Product използва оптимистично заключване (Optimistic Locking)[4]. Полето version с анотация @Version кара Hibernate автоматично да проверява дали версията на записа е променена от друга транзакция, и хвърля OptimisticLockException при конфликт[4].
Защитата срещу атаки за претоварване (Denial of Service) се осигурява чрез задължителна пагинация на продуктовия каталог, като максималният брой върнати записи в една страница е твърдо ограничен до 50 артикула[4]. JWT токените се издават с кратък срок на валидност, а паролите се съхраняват с BCrypt алгоритъм с алгоритмична сложност (Work Factor) не по-малка от 10[4].

# Спецификация на REST API ендпоинтите

Архитектурата на API слоя следва REST принципите за безстатусна комуникация[1]. Всеки ендпоинт връща стандартни HTTP кодове за успех и грешка[1].

| Метод | URL Маршрут | Минимални права | Описание на операцията | Код за успех |
|---|---|---|---|---|
| POST | /api/auth/register | Гост | Регистрация на нов потребител (BUYER или FARMER)[1]. | 201 Created[1] |
| POST | /api/auth/login | Гост | Вход и генериране на JWT токен[1]. | 200 OK[1] |
| GET | /api/auth/me | Логнат | Извличане на профила на текущо автентикирания потребител[1]. | 200 OK[1] |
| GET | /api/farms | Всички | Публичен списък с всички регистрирани ферми[1]. | 200 OK[1] |
| GET | /api/farms/{id} | Всички | Преглед на детайлен профил на конкретна ферма[1]. | 200 OK[1] |
| POST | /api/farms | FARMER | Първоначално създаване на профил на фермата[1]. | 201 Created[1] |

| | | | | |
|---|---|---|---|---|
| PUT | /api/farms/me | FARMER | Редакция на собствения фермерски профил[1]. | 200 OK[1] |
| GET | /api/categories | Всички | Извличане на продуктовите категории[1]. | 200 OK[1] |
| POST | /api/categories | ADMIN | Добавяне на нова категория от администратор[1]. | 201 Created[1] |
| GET | /api/regions | Всички | Извличане на номенклатурата с географски региони[1]. | 200 OK[1] |
| POST | /api/regions | ADMIN | Добавяне на нов географски регион[1]. | 201 Created[1] |
| GET | /api/products | Всички | Филтриран каталог (categoryId, regionId, month, search, page, size)[1]. | 200 OK[1] |
| GET | /api/products/{id} | Всички | Детайлен преглед на конкретен продукт[1]. | 200 OK[1] |
| GET | /api/products/my-products | FARMER | Извличане на продуктите на логнатия фермер за | 200 OK[1] |

| | | | | |
|---|---|---|---|---|
| | | | Dashboard[1]. | |
| POST | /api/products | FARMER | Публикуване на нов продукт[1]. | 201 Created[1] |
| PUT | /api/products/{id} | FARMER (Собственик) | Индивидуална редакция на продукт[1]. | 200 OK[1] |
| PUT | /api/products/bulk | FARMER (Собственик) | Масова пакетирана актуализация на цени и наличности[1]. | 200 OK[1] |
| DELETE | /api/products/{id} | FARMER (Собственик) | Изтриване на продукт[1]. | 204 No Content[1] |
| POST | /api/inquiries | BUYER | Изпращане на търговско запитване / заявка[1]. | 201 Created[1] |
| GET | /api/inquiries/sent | BUYER | Изглед на изпратените запитвания от купувача[1]. | 200 OK[1] |
| GET | /api/inquiries/received | FARMER | Изглед на получените запитвания към фермата[1]. | 200 OK[1] |
| PATCH | /api/inquiries/{id}/status | FARMER | Промяна на статус на запитване (ACCEPTED / REJECTED)[1]. | 200 OK[1] |

## Структура на JSON заявка за масова обработка (PUT /api/products/bulk)

При бързо обслужване на оперативните цени и наличности през интерфейса, Spring Boot контролерът приема списък от DTO структури и ги обработва в единна транзакция[1]:

```
JSON
[
  {
    "id": 101,
    "price": 4.50,
    "stockQuantity": 150.00,
    "isAvailable": true
  },
  {
    "id": 102,
    "price": 8.00,
    "stockQuantity": 0.00,
    "isAvailable": false
  }
]
```

## Спецификация на случаите на употреба (Use Cases)

Системата дефинира четири основни роли (актьори): Гост (неавтентикиран посетител), Купувач (клиент/ресторант), Фермер (производител) и Администратор (системен оператор)[3].

| ID | Заглавие | Основен актьор | Кратко описание | Засегнати релационни таблици |
|---|---|---|---|---|
| UC-01 | Регистрация и вход | Гост | Създаване на нов профил и получаване на JWT токен[3]. | users[3] |
| UC-02 | Управление на ферма | Фермер | Попълване и редакция на публичен профил за | farm_profiles, regions, users[3] |

| | | | | |
|---|---|---|---|---|
| | | | стопанството[3]. | |
| UC-03 | Публикуване на продукт | Фермер | Добавяне и редакция на единичен продукт, цена и мерни единици[3]. | products, categories, farm_profiles[3] |
| UC-04 | Дефиниране на сезонност | Фермер | Маркиране на месеците от годината, през които продуктът е наличен[3]. | product_seasonality, products[3] |
| UC-05 | Масова актуализация | Фермер | Пакетна промяна на цени и количества през Dashboard[3]. | products[3] |
| UC-06 | Търсене в каталога | Гост / Купувач | Търсене с филтриране по регион, категория, сезон и текст[3]. | products, farm_profiles, regions, categories, product_seasonality[3] |
| UC-07 | Изпращане на запитване | Купувач | Формулиране на B2B/B2C заявка за количество и съобщение[3]. | inquiries, users, products[3] |
| UC-08 | Обработка на заявки | Фермер | Преглед, приемане или отхвърляне на получените запитвания[3]. | inquiries[3] |

| | | | | |
|---|---|---|---|---|
| UC-09 | Поддръжка на номенклатури | Администратор | Управление на географските региони и категории[3]. | categories, regions[3] |

## Подробно описание на критични сценарии

### UC-03: Добавяне и редакция на нов продукт

При реализацията на сценария за добавяне на продукт, автентикираният фермер отваря екран за управление и попълва изискваните данни[3]. Процесът следва следните стъпки:

1. Фермерът избира категория от падащо меню, заредено от ендпоинта /api/categories, и въвежда наименование, описание, единична цена, мерна единица и текуща наличност[1].
2. Маркират се месеците на сезонност (напр. май, юни, юли)[3].
3. По избор се добавя URL адрес към изображение[3].
4. При изпращане на формата бекендът валидира данните с Bean Validation[3].
5. При липса на грешки Spring Boot извършва атомарен запис в таблицата products и добавя съответните записи в product_seasonality[2].
6. Системата връща отговор 201 Created с JSON структура на новия ресурс[1].

Ако въведената цена е отрицателна или липсва задължително поле, сервисният слой прихваща грешката и връща 400 Bad Request с детайлизирано описание на полетата, съдържащи невалидни стойности[3].

### UC-05: Масова актуализация на цени и наличности (Bulk Edit)

За осигуряване на бърз контрол върху ценообразуването, фермерът използва специализиран Dashboard[3]. Сценарият се изпълнява по следния начин:

1. Системата визуализира интерактивна таблица с всички публикувани от фермера продукти[3].
2. Полетата за цена, наличност и статус на активност са директно редактируеми[3].
3. След нанасяне на промени за множество артикули, потребителят задейства команда за запазване[3].
4. Фронтендът изпраща единствена HTTP PUT заявка с масив от обекти към /api/products/bulk[1].
5. Бекендът обработва списъка в рамките на единна @Transactional процедура[1].
6. При успешна валидация на всички елементи, промените се отразяват в базите данни и се връща код 200 OK[1].

При наличие на поне един невалиден обект в изпратения масив, целият пакет се отхвърля с откат на транзакцията, гарантирайки непроменено състояние на системата[4].

### UC-06: Филтриране и търсене в каталога

Посетителите и купувачите на платформата могат динамично да филтрират предлаганите стоки[3]. Потокът на изпълнение включва:

1. Потребителят задава комбинация от критерии: Регион = „Тракийска низина“

(regionId=1), Категория = „Млечни продукти“ (categoryId=3), Сезонен месец = „Май“ (month=5)[3].

2. Изпраща се HTTP GET заявка с Param параметри към /api/products?regionId=1&categoryId=3&month=5&page=0&size=12[1].
3. Spring Data JPA генерира SQL заявка с JOIN клаузи спрямо таблиците products, farm_profiles, categories и product_seasonality[2].
4. Резултатите се връщат под формата на пагиниран JSON обект (Page<ProductDTO>), съдържащ метаданни за общия брой страници и елементи[3].

## Тестови сценарии и осигуряване на качеството

За гарантиране на съответствието с изискванията е дефинирана матрица от тестови сценарии, покриваща позитивни, негативни, сигурностни и транзакционни тестове[5].

| ID | Област на изпитване | Описание на тестовия сценарий | Входни данни и предусловия | Очакван резултат | Вид тест |
|---|---|---|---|---|---|
| TS-01 | Автентикация | Успешна регистрация на потребител с роля FARMER | Валиден имейл, парола (Pass123!), име, телефон, role='FARMER'[5]. | Статус 201 Created, създаден ред в users с BCrypt хеширана парола[2]. | Позитивен[5] |
| TS-02 | Автентикация | Опит за вход с грешна парола | Съществуващ имейл и сгрешена парола[5]. | Статус 401 Unauthorized, без издаване на JWT токен[5]. | Негативен[5] |
| TS-03 | Продукти | Публикуване на нов продукт със сезонни месеци | Логнат FARMER, име, цена 5.50, наличност 100.00, месеци [5, | Статус 201 Created, нов запис в products и 3 записа в product_seasonality[2]. | Позитивен[5] |

| | | | | | |
|---|---|---|---|---|---|
| | | | 6, 7][5]. | | |
| TS-04 | Валидация | Добавяне на продукт с отрицателна цена | Логнат FARMER, цена -2.50, наличност 10.00[5]. | Статус 400 Bad Request, грешка от @DecimalMin, без промяна в БД[4]. | Негативен[5] |
| TS-05 | Сигурност | Опит за редакция на чужд продукт | Логнат FARMER A, изпраща PUT /api/products/{id_на_FARMER_B}[5]. | Статус 403 Forbidden, промените се блокират от Spring Security[4]. | Сигурност[5] |
| TS-06 | Bulk Edit | Успешна масова актуализация на цени и количества | Логнат FARMER, JSON масив с валидни стойности за 3 собствени продукта[1]. | Статус 200 OK, успешно обновени записи в базата данни[1]. | Позитивен[5] |
| TS-07 | Транзакции | Bulk Edit с присъствие на един невалиден обект | JSON масив за 3 продукта, като третият обект съдържа цена -10.00[5]. | Статус 400 Bad Request, пълен Rollback (нито един обект не се променя)[4]. | Транзакционен[5] |
| TS-08 | Каталог | Филтрирано търсене по регион, | GET /api/products?regionId= | Статус 200 OK, връща отсята | Позитивен[5] |

| | | | | | |
|---|---|---|---|---|---|
| | | категория и сезон | 1&categoryId=2&month=6&page=0&size=10[1]. | пагинирана колекция[1]. | |
| TS-09 | B2B Запитвания | Подаване на търговска заявка от купувач | Логнат BUYER, валидно product_id, количество 50.00 kg, съобщение[5]. | Статус 201 Created, нов запис в inquiries със статус PENDING[2]. | Позитивен[5] |
| TS-10 | Сигурност | Опит за изпращане на запитване към собствен продукт | Логнат FARMER, опит за запитване към продукт на неговата собствена ферма[4]. | Статус 400 Bad Request / 403 Forbidden, блокиране от сервизната логика[4]. | Негативен[5] |

# Архитектурна интеграция с React клиент и бъдещо развитие

Интеграцията между Spring Boot REST API и React SPA клиентското приложение налага решаване на ключови архитектурни въпроси, свързани с политиката за споделяне на ресурси между различни произходи (CORS), сигурното съхранение на JWT токени и управлението на асинхронните състояния.

## Конфигурация на CORS политиката в Spring Boot

Тъй като фронтенд приложението се обслужва от самостоятелен уеб сървър (напр. Vite dev server на port 5173 или Nginx на отделен домейн), браузърът прилага Same-Origin Policy. За да се позволи оторизиран пренос на данни, Spring Security се конфигурира с глобален CORS филтър:

```
Java
@Configuration
```

```
public class WebConfig implements WebMvcConfigurer

    @Override
    public void addCorsMappings(CorsRegistry registry)
        registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:5173", "https://farmer-catalog.bg")
                .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true)
                .maxAge(3600)
    }
}
```

## Модел за сигурност и управление на състоянието в React

1. **Съхранение на токени и Interceptors**: При успешна автентикация през /api/auth/login или /api/auth/google бекендът връща JWT[1]. За предотвратяване на Cross-Site Scripting (XSS) уязвимости, JWT токенът се съхранява в паметта на приложението (чрез React Context или Zustand store), докато Refresh токенът се предава през HttpOnly, Secure и SameSite=Strict бисквитка (Cookie). Интегриран Axios Interceptor автоматично прикачва Bearer токена към заглавната част Authorization на всяка HTTP заявка.
2. **Обработка на грешки и сесии**: При изтичане на валидността на токена бекендът връща код 401 Unauthorized[5]. Axios Interceptor-ът прихваща този отговор и инициира автоматично обновяване през ендпоинт за освежаване на сесията, или пренасочва потребителя към страницата за вход, изчиствайки локалното състояние.
3. **Управление на състоянието при Bulk Edit**: Екранът за масова актуализация на фермера се изгражда с помощта на React Hook Form или TanStack Table. Промените в цените и наличностите се отразяват първоначално в локалния state. При натискане на бутона за запазване се изпраща единична пакетна заявка към /api/products/bulk[1]. При успешен отговор от бекенда клиентското състояние се синхронизира без необходимост от пълно презареждане на страницата[1].

## Процес на социален вход чрез Google OAuth 2.0 (SSO Flow)

Интеграцията на Google вход в React използва Authorization Code Flow with PKCE или клиентска библиотека @react-oauth/google. След като потребителят се автентикира пред Google, клиентското приложение получава id_token и го препраща към бекенд ендпоинта /api/auth/google. Spring Boot валидира токена, проверява дали съществува такъв потребител в таблицата users и генерира стандартна JWT сесия[1]. Това осигурява единен модел на оторизация независимо от избрания метод за вход[1].

# Заключение и системни препоръки

Изготвената софтуерна спецификация за платформата „Фермерски каталог“ дефинира архитектура за изграждане на REST API с Java и Spring Boot[1]. Преминаването към

нормализиран релационен модел с 7 таблици в MySQL осигурява висока скорост при филтриране по региони, категории и сезонни месеци, като същевременно гарантира транзакционен интегритет[2].
Разделението на роли (BUYER, FARMER, ADMIN), въведените DTO валидации, контролът върху ресурсното собственост и оптимистичното заключване създават среда, защитена от грешки при паралелна работа и опити за неоторизиран достъп[1]. Проектираният интерфейс за масова актуализация (PUT /api/products/bulk), поддръжката на B2B/B2C запитвания и подготвената съвместимост с React SPA и Google OAuth 2.0 предоставят основа за успешното развитие и производствено внедряване на софтуерната система[1].

**Цитирани творби**


1. API Ednpoints Sample.docx
2. DB Tables.docx
3. Use Cases.docx
4. Project Constraints.docx
5. Test Cases.docx